



Computer Vision and Image Processing

Lecture 08

Thresholding

Thresholding is a fundamental segmentation technique used to convert a grayscale image into a binary image. It separates foreground (object) from background based on grey-level intensity.

Thresholding Techniques:

1. Global Thresholding
2. Adaptive (Local) Thresholding
3. Otsu's Method (Automatic Global Thresholding)

Global Thresholding

Global thresholding uses a **single threshold value (T)** for the entire image.

When to use it

- Image has **uniform illumination**
- Background and foreground intensities are clearly separated

How it works

- Choose a threshold T manually or automatically.
- Compare all pixel values to this T .
- Output is binary.

Global Thresholding

Algorithm

- Compute histogram
- Pick a threshold
- Apply binary formula

Example

- If $T = 120$:
Pixels $> 120 \rightarrow$ white (1)
Pixels $\leq 120 \rightarrow$ black (0)

Global Thresholding

Pros

- Very fast
- Simple to implement

Cons

- Fails in uneven/poor lighting
- Sensitive to shadows and highlights

Adaptive (Local) Thresholding

Instead of one global threshold, **each pixel gets its own threshold** based on its **local neighbourhood**.

Useful when:

- There is **non-uniform illumination**
- Background varies across the image
- Shadows or bright regions exist

Adaptive (Local) Thresholding

How it works

Divide the image into small windows (e.g., 15×15). For each window:

Compute a threshold using:

- Mean
- Weighted mean (Gaussian)
- Median

Then threshold that region separately.

Adaptive (Local) Thresholding

Algorithm

- For each pixel at (x, y):

$$T(x, y) = \text{mean of neighbourhood} - C$$

Pros

- Works well in varying illumination
- Better for document images, medical images

Cons

- More computationally expensive
- Window size selection is critical

Otsu's Method (Automatic Global Thresholding)

Otsu's method, named after its inventor Nobuyuki Otsu, is one of many binarization algorithms.

Otsu's Method works best with Bimodal Histogram.

A **bimodal histogram** is a histogram of image grey levels that contains **two distinct peaks (modes)**. These peaks represent **two dominant intensity groups** in the image, usually corresponding to **foreground** and **background**.

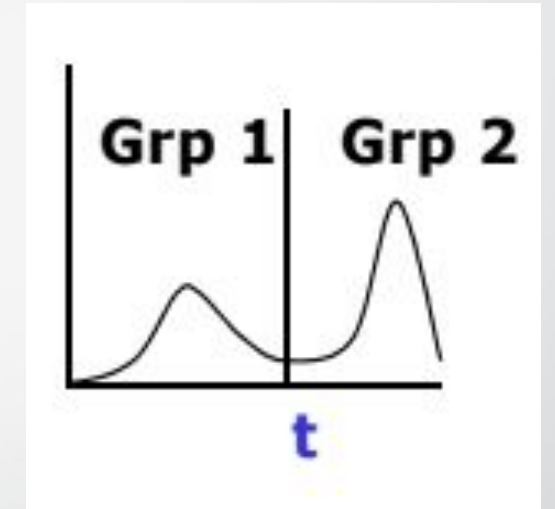
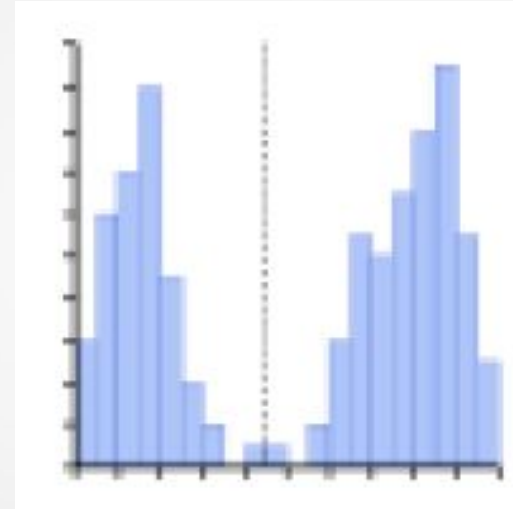
Otsu's Method (Automatic Global Thresholding)

A **bimodal histogram** has:

- Two **major peaks**
- A **valley** between them
- Two **clusters** of pixel intensities

This usually occurs in images where:

- The **foreground object** has intensities very different from the **background**
- The image contains **two primary classes** of brightness



Otsu's Method (Automatic Global Thresholding)

1. Within-Class Variance (How "Pure" are the baskets?)

- Imagine you draw a cutoff line and throw your clothes into the two baskets.
- Now, look *inside* the "Lights" basket. Are they all exactly the same shade of pure white? Probably not. You might have some eggshell, some light gray, and maybe a pale-yellow shirt. The differences between the clothes *inside this single basket* is the **Within-Class Variance**.
- **High Within-Class Variance:** You did a bad job sorting. Your "Lights" basket has dark red shirts in it, and your "Darks" basket has white socks in it. The colors are all over the place.
- **Low Within-Class Variance:** You did a great job. The "Lights" basket is entirely pale colors, and the "Darks" basket is entirely dark colors.

Otsu's Goal: Make the "spread" of colors inside each basket as tiny as possible.

Otsu's Method (Automatic Global Thresholding)

2. Between-Class Variance (How "Different" are the two baskets from each other?)

- Now, step back and look at the two baskets as a whole. Ignore individual socks; just look at the **average color** of Basket 1 and the **average color** of Basket 2.
- **Low Between-Class Variance:** The average color of Basket 1 is medium-gray, and the average color of Basket 2 is slightly-darker-gray. They look too similar. Your cutoff line was in the wrong place.
- **High Between-Class Variance:** The average color of Basket 1 is bright white, and the average color of Basket 2 is pitch black. They are drastically different from each other.
- **Otsu's Goal:** Push the average color of the two baskets as far apart as possible.

Otsu's Method (Automatic Global Thresholding)

Pros

- Fully automatic
- Works well for bimodal images

Cons

- Fails if lighting is uneven
- Fails if histogram is not bimodal (e.g., multimodal or low contrast)

Neighborhoods in Binary Images

- In digital image processing, a **binary image** is an image where each pixel can only have one of two values: typically, **0** (representing the black background) or **1** (representing the white foreground).
- When analyzing these images to find shapes, boundaries, or connected components, computers need a mathematical way to look at a pixel's relationship with the pixels immediately surrounding it. This localized surrounding area is called a pixel's **neighborhood**.
- If we take a target pixel **p** located at coordinates **(x, y)** on an image grid, there are three primary ways to define its neighbors:

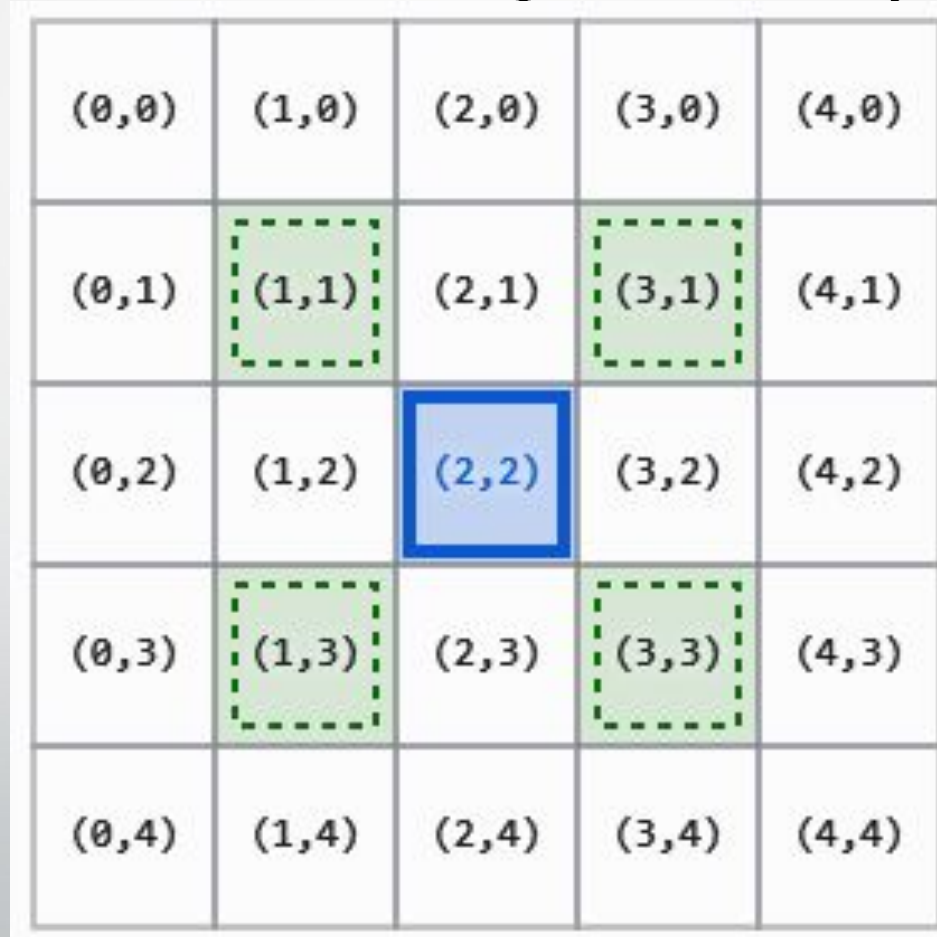
1. The 4-Neighborhood (N4)

- The 4-neighborhood consists only of the pixels that share a direct, flat edge with the center pixel p . These are the pixels directly above, below, to the left, and to the right

$(0,0)$	$(1,0)$	$(2,0)$	$(3,0)$	$(4,0)$
$(0,1)$	$(1,1)$	$(2,1)$	$(3,1)$	$(4,1)$
$(0,2)$	$(1,2)$	$(2,2)$	$(3,2)$	$(4,2)$
$(0,3)$	$(1,3)$	$(2,3)$	$(3,3)$	$(4,3)$
$(0,4)$	$(1,4)$	$(2,4)$	$(3,4)$	$(4,4)$

2. The Diagonal Neighborhood (ND)

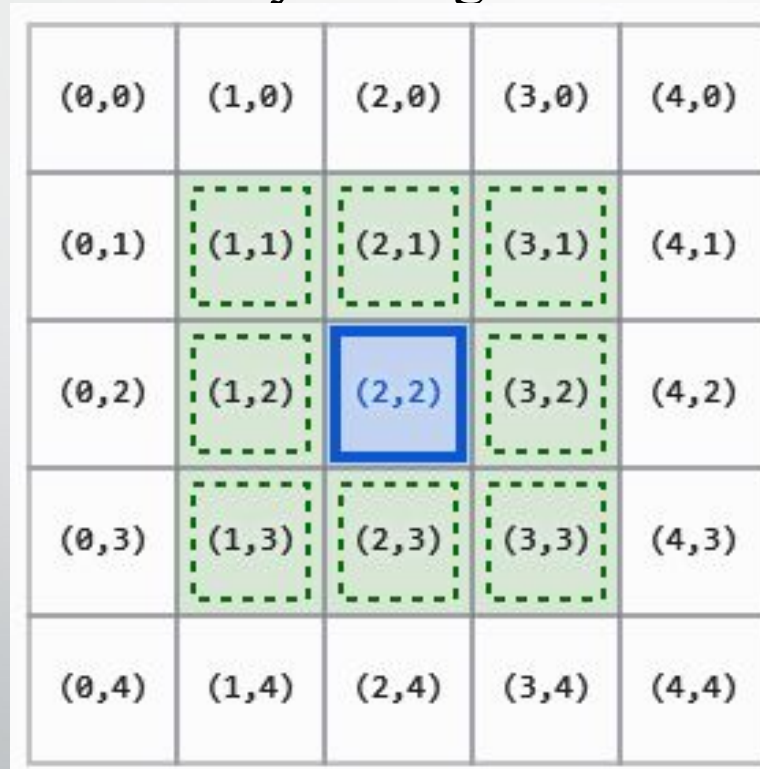
- The diagonal neighborhood consists of the four pixels that only share a corner with the center pixel p .



(0,0)	(1,0)	(2,0)	(3,0)	(4,0)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)
(0,4)	(1,4)	(2,4)	(3,4)	(4,4)

3. The 8-Neighborhood (N8)

- The 8-neighborhood is simply the combination of the 4-neighborhood and the diagonal neighborhood. It includes every single pixel that touches the center pixel p in any way, whether by an edge or a corner.



(0,0)	(1,0)	(2,0)	(3,0)	(4,0)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)
(0,4)	(1,4)	(2,4)	(3,4)	(4,4)

Neighborhood Ambiguity

- The concept of **neighborhood ambiguity**—often called the **Connectivity Paradox**—is one of the most famous quirks in digital image processing.
- It happens because a grid of square pixels doesn't behave perfectly like continuous space. In the real world, if you draw a closed circle, it clearly separates the "inside" from the "outside." In a pixel grid, depending on how you define your neighborhoods, that logic can break down.
- Here is how the ambiguity happens, specifically when dealing with diagonal lines.

Neighborhood Ambiguity

- Imagine a simple 2x2 block of pixels where the top-right and bottom-left pixels are **black** (foreground), and the top-left and bottom-right are **white** (background).
- The ambiguity arises when we try to answer a simple question: **Are the black pixels connected to each other? And are the white pixels connected to each other?**

Neighborhood Ambiguity

If we apply the same neighborhood rules to both the foreground and the background, we run into logical paradoxes:

- **Scenario A: Everything is 4-Connected** If we decide that pixels only connect via flat edges (4-neighborhood), then the two black pixels are **not** connected. But simultaneously, the two white pixels are also **not** connected. The black line is broken, but the white space can't flow through it either.
- **Scenario B: Everything is 8-Connected (The Paradox)** If we decide that pixels connect via corners too (8-neighborhood), then the two black pixels **are** connected. They form a diagonal line. However, the two white pixels are *also* connected through the exact same corner point. This means a continuous background line crosses completely through a continuous foreground line without breaking it—a geometric impossibility in normal space.

Neighborhood Ambiguity

The Standard Solution

- To fix this ambiguity and ensure that a closed loop actually traps the background inside it, image processing software usually uses a mixed rule: **We define the foreground objects as 8-connected, and the background space as 4-connected (or vice versa).** By doing this, the black pixels connect diagonally to form a solid boundary, and the white background cannot squeeze through the corners, keeping the "inside" strictly separated from the "outside."



The End